

AgenticAPKFlow

AI-Powered Agentic Workflow for
Automated Android APK Security Analysis

Agno Framework

FastMCP

ReAct Reasoning

MITRE ATT&CK

The Problem with Traditional APK Scanners

Why rule-based tools fall short

✗ Rule-Based Scanners

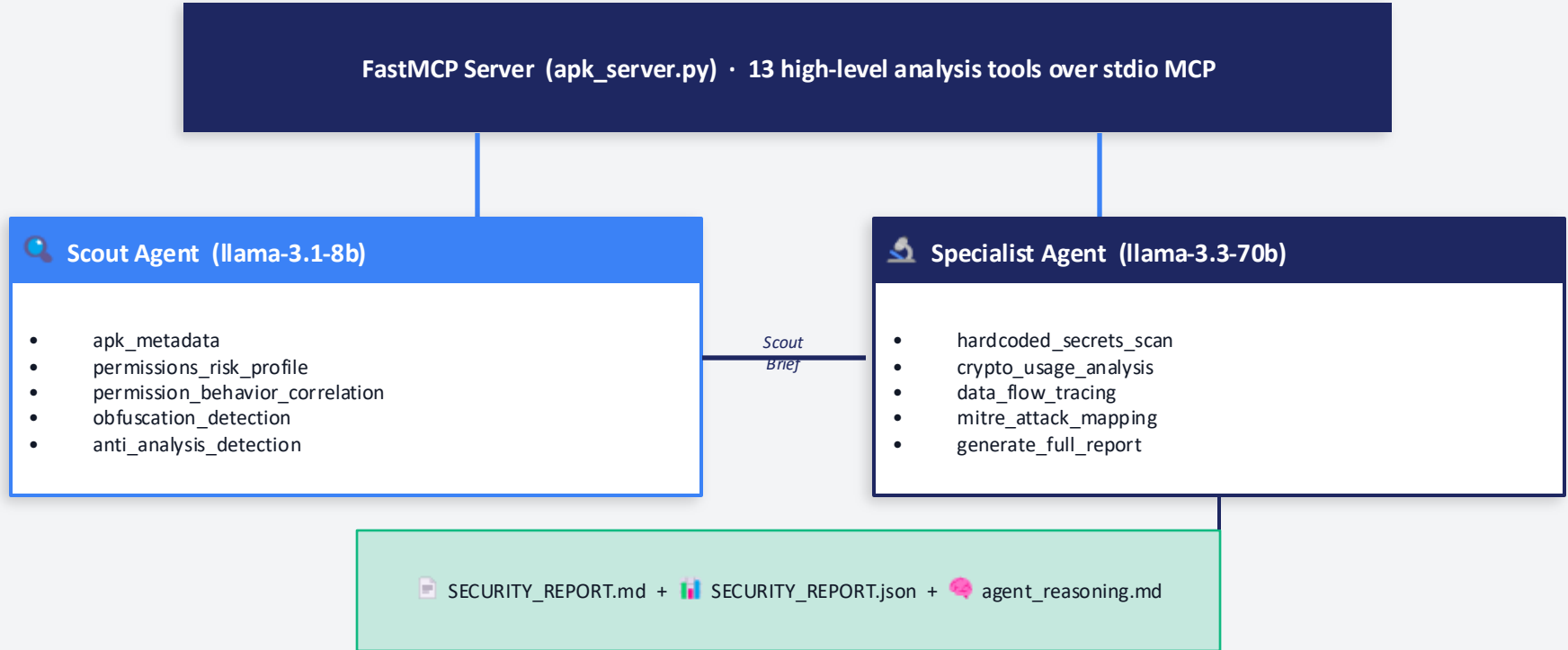
- Scan every file — no context awareness
- High false-positive rate (noise)
- Cannot correlate permission + behavior
- Miss native .so crypto patterns
- No hypothesis-driven investigation
- Static rules, no adaptive reasoning

✓ AgenticAPKFlow

- Hypothesis-driven selective analysis
- Evidence-backed findings only
- Permission × behavior correlation
- Native crypto detection + warning
- True ReAct reasoning loop
- Scout + Specialist agent collaboration

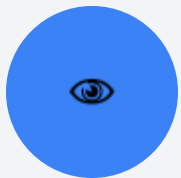
System Architecture

Multi-agent ReAct workflow via FastMCP



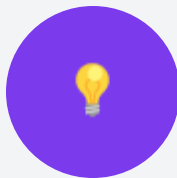
True ReAct Reasoning

Agents decide tool invocation order based on findings



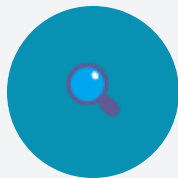
OBSERVE

Extract metadata
& permissions



HYPOTHESIZE

Form threat
hypotheses



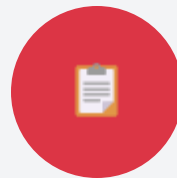
INVESTIGATE

Run targeted
tools



VALIDATE

Confirm or
refute findings



CONCLUDE

Final risk
verdict

Example — InsecureBankv2 Specialist Reasoning:

- ① Scout found READ_CONTACTS + INTERNET → "Hypothesis: contact exfiltration"
- ② hardcoded_secrets_scan found 5 secrets → "Running crypto_usage_analysis to understand key management"
- ③ AES/CBC + MD5 found → "Checking network_behavior_analysis for exfiltration endpoints"
- ④ 56 URLs, 1 hardcoded IP → "Tracing data flows to confirm Crypto_Key→Network paths" → CONFIRMED

High-Level Tool Design

13 MCP tools — structured risk-assessed output, not raw command wrappers

permissions_risk_profile

Scout

25-entry DB → HIGH/MEDIUM/LOW per permission

permission_behavior_correlation

Scout

Risk Correlator: confirms hypotheses via API usage evidence

hardcoded_secrets_scan

Specialist

10 pattern types: AES keys, API tokens, Base64 blobs

crypto_usage_analysis

Specialist

ECB mode flag, native .so crypto warning, key tracing

trace_data_flows

Specialist

Logic Tracer: source→sink co-occurrence (SMS, GPS, Contacts)

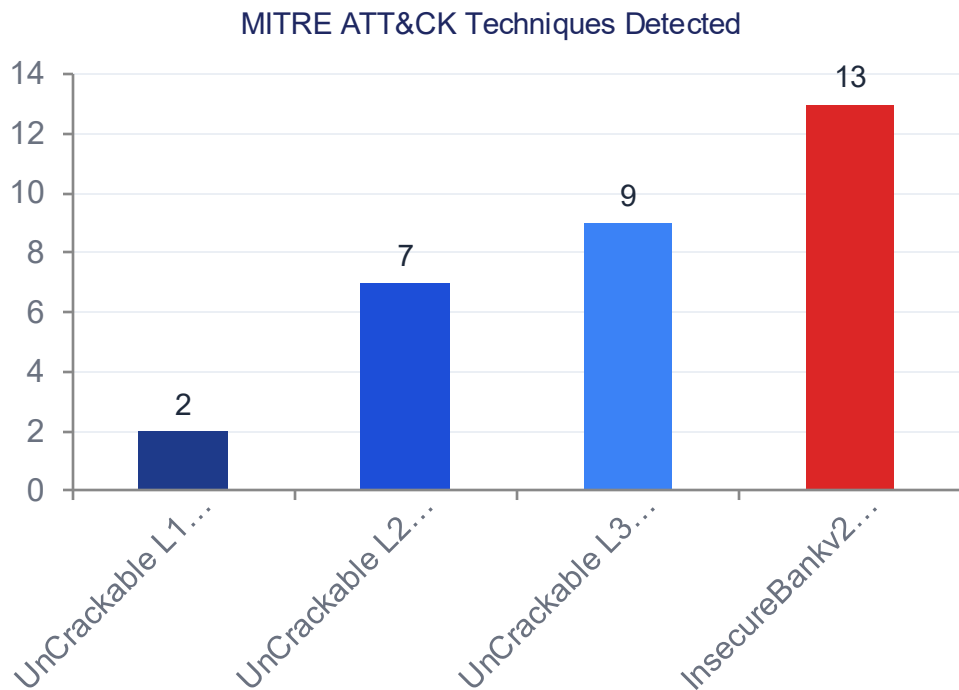
mitre_attack_mapping

Specialist

Maps to 15 MITRE ATT&CK Mobile techniques with evidence

Evaluation Results

4 APK samples of increasing complexity



APK	Secrets	Confirmed	Anti-Analysis
UnCrackable L1	2	0	Root Detection
UnCrackable L2	1	0	Debugger
UnCrackable L3	1	0	Frida+Root+Debug
InsecureBankv2	5	3 ✓	Timing+Root



InsecureBankv2 Highlights

- 13 MITRE techniques across 5 tactics
- 3 confirmed threats with API evidence
- 56 URLs · 18 domains · 1 hardcoded IP

Confirmed Threat Hypotheses

InsecureBankv2 — evidence-backed findings

Contact Harvesting

T1636.003

✓ CONFIRMED

Hypothesis: READ_CONTACTS + INTERNET = contact list exfiltration

Evidence: `getContentResolver()` detected in source

Toll Fraud

T1616

✓ CONFIRMED

Hypothesis: SEND_SMS = silent premium-rate SMS messages

Evidence: `SmsManager.sendTextMessage()` detected

Dynamic Code Loading

T1407

✓ CONFIRMED

Hypothesis: INTERNET + WRITE_STORAGE = download & execute DEX

Evidence: `DexClassLoader` detected in 10 files

Dynamic Analysis: CAPEv2 Integration

Hybrid static + dynamic analysis with sandbox behavioral data

⚡ CAPEv2 Sandbox Extracts

Runtime API calls

30+ suspicious APIs mapped to risk categories

Live network IOCs

TCP connections, HTTP POST, DNS queries

Process injection

VirtualAllocEx→WriteProcessMemory→CreateRemoteThread

Registry persistence

Run keys written for auto-start

Dropped files

Executables/payloads created on disk

Malware family

CAPE classification (e.g. Emotet)

🔗 Unified MITRE ATT&CK Table

ID	Source	Technique
T1406	Obfuscation	 Static
T1521.001	Sym. Crypto C2	 Static
T1055	Process Injection	 Dynamic
T1547.001	Registry Run Keys	 Dynamic
T1071	App Layer Protocol	 Dynamic
T1497	Sandbox Evasion	 Dynamic

Tech Stack & Deployment

Fully containerized · Docker + docker-compose · start.sh validates dependencies

Agent Framework

Agno

Multi-agent orchestration

MCP Protocol

FastMCP 3.1.1

13 tools over stdio

LLM (Scout)

LLaMA 3.1 8B

via Groq API

LLM (Specialist)

LLaMA 3.3 70B

via Groq API

Decompiler

JADX 1.5.0

APK → Java source

Container

Docker

docker-compose + start.sh

```
$ docker compose run --rm agentic-apk /app/data/InsecureBankv2.apk
```

```
# No LLM needed:  docker compose run --rm agentic-apk python3 src/agent/direct_run.py /app/data/app.apk
```

Limitations & Future Work

Honest assessment and next steps

Current Limitations

- Static analysis only — dynamic behaviors not captured (runtime decryption, C2)
- Native .so libraries detected but not analyzed (needs Ghidra/radare2)
- False positives from Android support library code (accessibility strings)
- Free LLM quota limits (100k tokens/day on Groq) — `direct_run.py` fallback exists
- No MobSF comparison (research component not completed due to time)

Future Work

- Dynamic analysis via Frida / CAPEv2 integration for behavioral detection
- Native binary analysis tools (Ghidra scripting via FastMCP)
- MobSF comparison on Ghera benchmark (60 vulnerability test cases)
- Extend to PE/ELF binary analysis for full binary exploitation workflow
- Cross-session agent memory to learn from previously analyzed samples

Conclusion

- ✓ True ReAct reasoning — agents autonomously decide tool invocation order
- ✓ 13 FastMCP tools with structured risk-assessed output (not raw wrappers)
- ✓ Confirmed threats with API evidence on real-world banking malware
- ✓ MITRE ATT&CK mapping: 2 → 7 → 9 → 13 techniques (scales with complexity)
- ✓ Fully Docker containerized with dependency validation
- ✓ Dual output: Markdown report + structured JSON

github.com/ELANSARI05/AgenticAPKFlow